Milestone Four

Database Artifact

Matthew Schaub

SNHU

CS 499

06/07/2026

The artifact for databases is my CS 340 Grazioso Salvare animal shelter dashboard. This project was created using Python, MongoDB, PyMongo, Dash, Plotly, and Dash Leaflet. The dashboard connects to the AAC shelter outcomes data and allows users to filter animal records by rescue profiles. The application then displays the records in a table, shows a breed chart, and updates the map based on the selected animal.

I selected this artifact for my ePortfolio because it is a working database application and not just a small script. It shows how a program can connect to a database, retrieve records, filter data, and display the results in a way that helps a user make a decision. This makes it a good artifact for the database category because the main value of the project comes from the MongoDB data and how the dashboard uses that data. This also fits my career goals in automation and digital solutions because industrial systems use dashboards, databases, and reporting tools to give teams better visibility into production and plant information.

The artifact was improved in several ways. One of the main improvements was creating a stronger database connection layer. In the original version, the dashboard worked, but it used hardcoded database information and pulled more data than it really needed. In the enhanced version, the MongoDB username, password, host, database, and collection are loaded from environment variables and a local .env file. This means the password does not have to be written directly inside the Python source code. This was important because this artifact may be uploaded to GitHub as part of my ePortfolio, and database passwords should not be exposed in a public project.

Another security improvement was adding better password management to the setup process. I added a setup script that can prompt for a MongoDB dashboard password using hidden input. The script can also generate a strong random password if the user does not want to create

one manually. I added a password rotation script so the password can be changed later without searching through the whole program. The dashboard uses a limited MongoDB user for the aac database instead of relying on a broader administrative account. I also added a .gitignore file to protect the local .env file from being committed to GitHub.

The database logic was also improved. The enhanced version uses approved rescue profile filters instead of allowing raw query input. This makes the dashboard safer because the user can only choose from known filter options. I also added projections so the dashboard only receives fields that are needed for the table, chart, and map. I added paging so the dashboard does not need to load every record at one time. I also added indexes for fields used often in the rescue filters. Another improvement was moving more of the chart work into MongoDB. The breed chart now uses an aggregation pipeline to group and count breed information before the results are sent back to the dashboard.

In Module One, I identified course outcomes that my enhancements would align with. This enhancement makes strong progress toward course outcome 1 because the dashboard helps support decision making. Course outcome 2 was met through the narrative, code comments, README information, and setup instructions. Course outcome 3 had progress been made because I had to make design choices and think through trade-offs. One trade-off was deciding what should happen in the database and what should happen inside the dashboard. Using MongoDB aggregation for the breed chart reduces the amount of client-side processing. Course outcome 4 is demonstrated by my use of Python, MongoDB, PyMongo, Dash, Plotly, Dash Leaflet, indexes, projections, aggregation pipelines, and environment-based configuration. Course outcome 5 is demonstrated by the security improvements. I removed hardcoded credentials, added private password storage through the .env file, added password generation and

rotation, used a limited database user, protected the .env file from GitHub, and limited dashboard queries to approved filter options. An update on my original plan is that I added more security work than I first expected. My original database enhancement plan focused on environment variables, projections, indexes, approved filters, and aggregation pipelines. As I worked on the project, I realized that password management was one of the most important improvements because this artifact could be viewed in a public setting(ePortfolio).

This process taught me that a program can work for a class project and still need changes before it is ready to show as a professional artifact. When I first created the CS 340 dashboard, my main goal was to connect to MongoDB and make the table, chart, and map work. Reviewing it now helped me look at the project more like a soon to be professional. I had to think about what would happen if another person tried to run the program, if the data set became larger, or if the project was uploaded to GitHub with private information still inside it.

The biggest challenge was making the dashboard more secure while still keeping it easy to run locally. A hardcoded password is simple, but it is not a good habit. Using a .env file is better, but it means the setup instructions need to be clear. Another challenge was making database improvements without changing the whole purpose of the dashboard. I wanted to keep the original rescue profile idea, table, chart, and map, but improve the way the project connects to and uses the database. Overall, these enhancements made the artifact stronger for my ePortfolio because it now shows database development, dashboard design, secure configuration, and better professional coding practices.